TANDEM

Tiered, Adaptive, Near-memory Data-and-compute placement for Efficient Multimodal KV caches

Where should attention run? A hypothesis-driven simulation study on real vision-language attention

Sudarshan Sudhakar

Independent research. Code, traces and all result tables: `KV-Tier` repository

Why "TANDEM". The system makes two runtime decisions *in tandem*: where each KV entry *lives* (data placement) and where each read is *computed* (computation placement).

Contributions.

1. A formulation of KV-cache placement as a *joint data- and computation-placement* problem, and two SimPy simulators of it: an access-cost model (E1–E7) and a system model with finite compute-in-memory (CIM) units, host lanes, a shared bus, work packets, migration traffic and a compute scheduler (E8–E9) that reduces exactly to the first (unit-tested).
2. A negative result with a diagnosis: the synthetic-trace win does not replicate on real attention, and the win that appears at per-(token, layer) granularity is *not* a layer or role prior (E2–E4).
3. A pre-specified scale-up (42 images, 3 image kinds, 2 model sizes, leave-one-trace-out, bootstrap CIs) establishing the data-placement saving and its failure region (E5).
4. Two checks on the strongest objections to a simulation study: non-oracle importance (E6) and measured output change when INT4 is applied in the real model (E7).
5. Evidence that the compute-site decision is first-order under contention, including a correction of an earlier latency result that was an artefact of serial accounting, and a bottom-up migration cost of ≈ 1.05 HOT accesses in place of the assumed 10× (E8).
6. Evidence that placement must depend on data reuse (reusable, streamed, random), with pre-specified hypotheses of which two fail narrowly and are reported as such (E9).

Key results at a glance.

- −22.3% / −23.4% **energy** (SmolVLM-256M / 500M) from dynamic data placement vs static modality tiering, at neutral quality, on 42 images; thresholds transfer between sizes; break-even at ∼50× migration cost; **fails under sparse reads** (+5.7% at top-10%).
- Robust to a degraded importance signal (−18.6 to −21.8%) but **not** to one score per token (−7.6%). With real INT4 in the model: output KL **21.8% lower** than static and **9.4× lower** than random placement.
- Under contention (E8): tiering moves **11.8× fewer bytes** at **0.64× the makespan** of a flat memory (the serial model's +34% latency was an artefact). Always computing in memory is up to **5× slower** than flat with 2 CIM units; an **adaptive spill scheduler beats both** fixed choices.
- Under reuse (E9): −27.4% energy for reusable data; inert for streamed data; for random access up to +82% **energy** at 10× migration cost unless migrations are priced against observed reuse. The reuse gate leaves +1 to +2% and **misses our pre-specified criterion**.

Abstract

A memory-centric architecture should decide where computation runs from the location, movement, reuse and error tolerance of the data, and the run-time cost of making that decision must be shown to be worth paying. We study this on the key–value (KV) cache of vision-language models (VLMs), whose decode-time reads dominate memory traffic. The memory has a HOT tier (FP16 DRAM/HBM, host compute) and a WARM tier (INT4 compute-in-memory). A runtime makes two decisions from live importance and reuse estimates: *data placement* (which tier an entry occupies, changed by a paid migration) and *computation placement* (whether a WARM read runs in memory or on the host). The study is a chain of nine experiments on real SmolVLM attention (42 images, two model sizes) in which each result motivates the next and failures are kept: a synthetic-trace win did not replicate on real attention, and a layer-prior explanation was falsified. With dense reads and a 10× migration cost, dynamic data placement lowers energy by **22.3%** (256M) and **23.4%** (500M) against static modality tiering at neutral quality, breaks even near 50× migration cost, and vanishes under sparse reads. It survives noisy, stale or sampled importance, and with INT4 applied inside the model it changes the next-token distribution 21.8% less than static tiering and 9.4× less than random placement. A system-level model with finite CIM units, host lanes and a shared bus then shows that tiering moves 11.8×

Table 1: How this study instantiates the memory-centric placement problem.

Requirement	In this work
Placement depends on location, movement, reuse, data characteristics	Location = tier of each entry; movement = bytes on the shared bus (reads, spills, partial sums, migrations); reuse = observed read frequency; characteristics = attention importance and tolerance to INT4 error.
Compute near data when beneficial, conventional compute otherwise	WARM reads run on CIM units, HOT reads on host lanes; a scheduler may spill WARM reads to the host (Sec. 5.8).
Memory, compute, movement, placement and runtime decisions interact explicitly	SimPy model with finite bus channels, host lanes and CIM units, work packets, a per-step barrier, and migration traffic that competes with reads (Sec. 3.6).
Feasibility and overhead of dynamic placement	Migration cost derived bottom-up and swept to 50× (E5, E9b); estimator imperfection (E6); contention, spill and utilisation (E8).
Different data need different placement	Reusable, streamed and randomly accessed reads, on real attention (E9); sparse top-k reads (E5).

fewer bytes than a flat memory at 0.64× its makespan (the serialised model's +34% latency was an artefact); that always computing in memory is up to 5× slower than a flat memory when CIM units are scarce; and that an adaptive scheduler that spills to the host beats both fixed choices on makespan. Finally, the right placement depends on reuse: dynamic placement saves 27% energy for reusable data (bottom-up migration cost), is inert for streamed data, and under random access loses up to +82% energy unless migrations are priced against observed reuse. A reuse gate removes most but not all of that loss (+1 to +2%), narrowly missing our pre-specified parity criterion.

Keywords: memory-centric computing · computation placement · compute-in-memory · KV cache · vision-language models · data movement · negative results.

1 Introduction

In a memory-centric architecture the question is not only *where data lives* but *where the computation on it runs*: next to the data, in a compute-in-memory (CIM) array, when that avoids moving bytes, and on conventional compute when that is faster or more accurate. The right choice depends on properties of the data that change at run time: how often it is reused, whether it is streamed through once or accessed at random, and how much error a computation on it can tolerate. A fixed rule cannot be right for all of these, so the placement must be decided by the runtime, and the decision itself (observing, migrating data, scheduling work, contending for shared resources) has an overhead that must be shown to be worth paying.

We study this problem on a concrete, data-intensive workload: the key–value (KV) cache of a vision-language model (VLM). A VLM prefills an image as hundreds to thousands of visual tokens; each token adds a KV entry in every transformer layer, and every generated token reads that cache again. For long multimodal contexts the cache, not the arithmetic, sets the energy and bandwidth bill. A large body of recent work shrinks the cache by pruning, quantizing or merging visual tokens [1, 2, 3, 4, 5], and all of it assumes a flat memory and decides only *what to discard*. We ask the architectural question instead (Table 1 states how the workload instantiates the general problem). The memory has a HOT tier (conventional DRAM/HBM, FP16, lossless; reads are computed on the host) and a WARM tier (INT4 CIM arrays: lower energy per byte, slower, lossy; reads are computed in memory). A runtime makes two decisions: *data placement* (which tier each (token, layer) entry occupies, changed by paying for a migration) and *computation placement* (whether a read of WARM-resident data runs in memory or is pulled to the host). Both decisions interact through finite CIM units, host lanes and a shared bus, and both are driven by live observations of importance and reuse.

Approach. We build SimPy simulators with explicit models of access cost, quantization error, tier moves and (for E8–E9) compute resources and data movement, and we feed them *real* decode-time attention extracted from SmolVLM. We treat the work as a sequence of falsifiable hypotheses. Each experiment is motivated by the weakness of the previous one (Fig. 1). Selection rules and success criteria were written in the lab notes before the corresponding runs (a self-imposed discipline, not an external registration), and every deviation from them is logged.

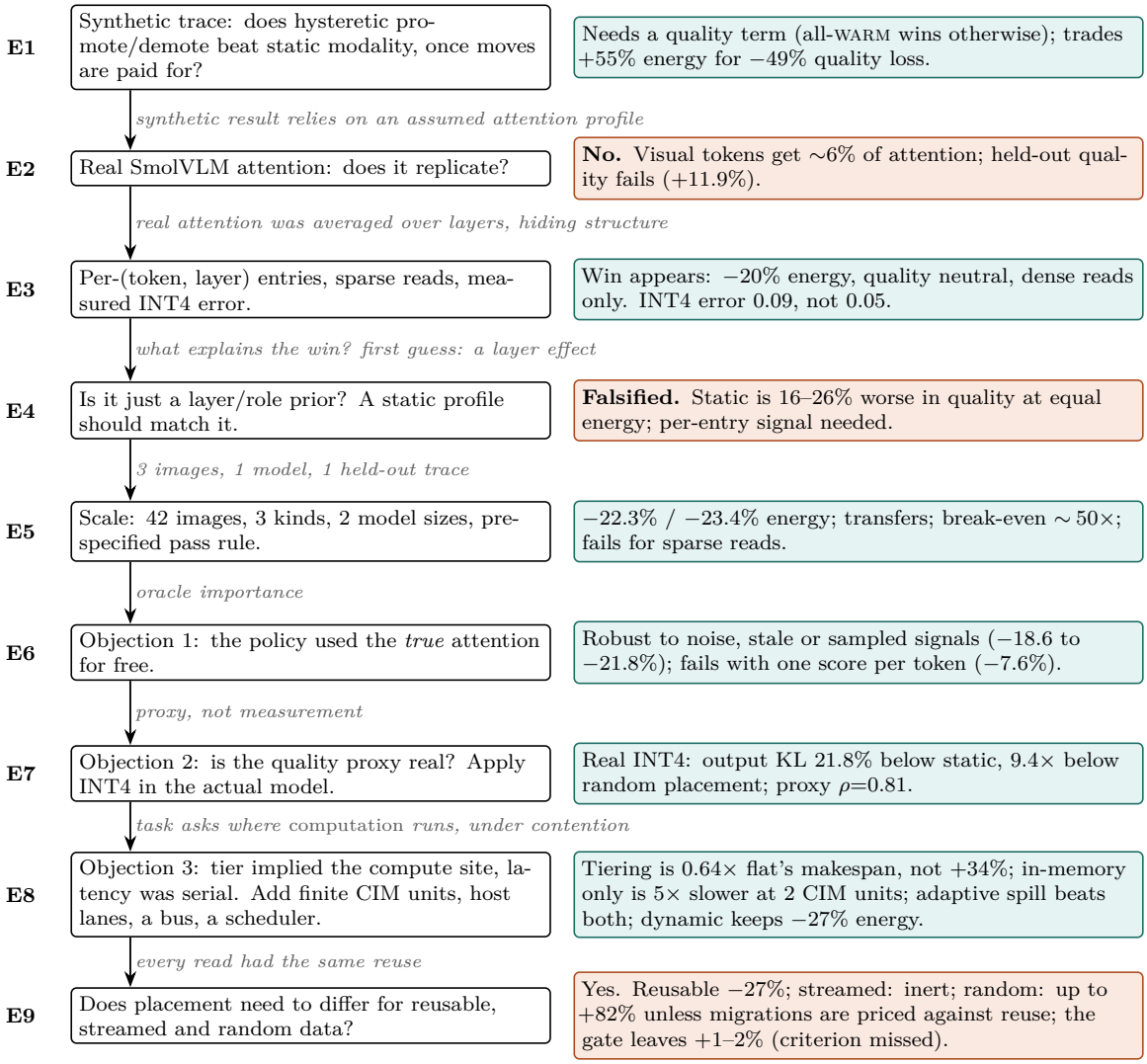


Figure 1: The study as a chain of hypotheses. Each box states the question, the finding, and (on the arrow) the weakness that forced the next experiment. Red marks results that overturned an earlier belief (E2, E4) or missed a pre-specified criterion (E9).

Contributions and name. The contributions and the key results are summarised in the box on the first page. We call the study TANDEM because its two runtime decisions, data placement and computation placement, are made in tandem and interact through shared resources.

2 Related Work

Algorithm-level visual KV compression. VL-Cache [1] (modality- and sparsity-aware budgeting), ZipVL [2] (dynamic token sparsification), AirCache [3] and TopV [4] (inter-modal relevance and token pruning) and FastV [5] (early-layer visual token pruning) show that visual tokens are, on average, less important to decoding than text tokens and can be compressed with small accuracy loss. For text LLMs, H2O [6] and SnapKV [7] keep heavy-hitter entries, StreamingLLM [8] keeps attention sinks plus a recent window, KIVI [9] quantizes the cache to 2 bits, and Quest [10] reads only query-relevant pages. These methods *discard or shrink*. We instead ask where an entry should reside and what it costs to move it, a question they do not pose because their memory is flat.

Memory-centric and in-memory computing. The case for computing where the data is, because moving data dominates energy, is made in the processing-in-memory literature [12], and commercial DRAM-based PIM exists [13]. Recent proposals place the attention stage of LLM decoding on in-/near-memory units [14, 15]. We do not compare against these designs. Our question is the runtime one they leave open

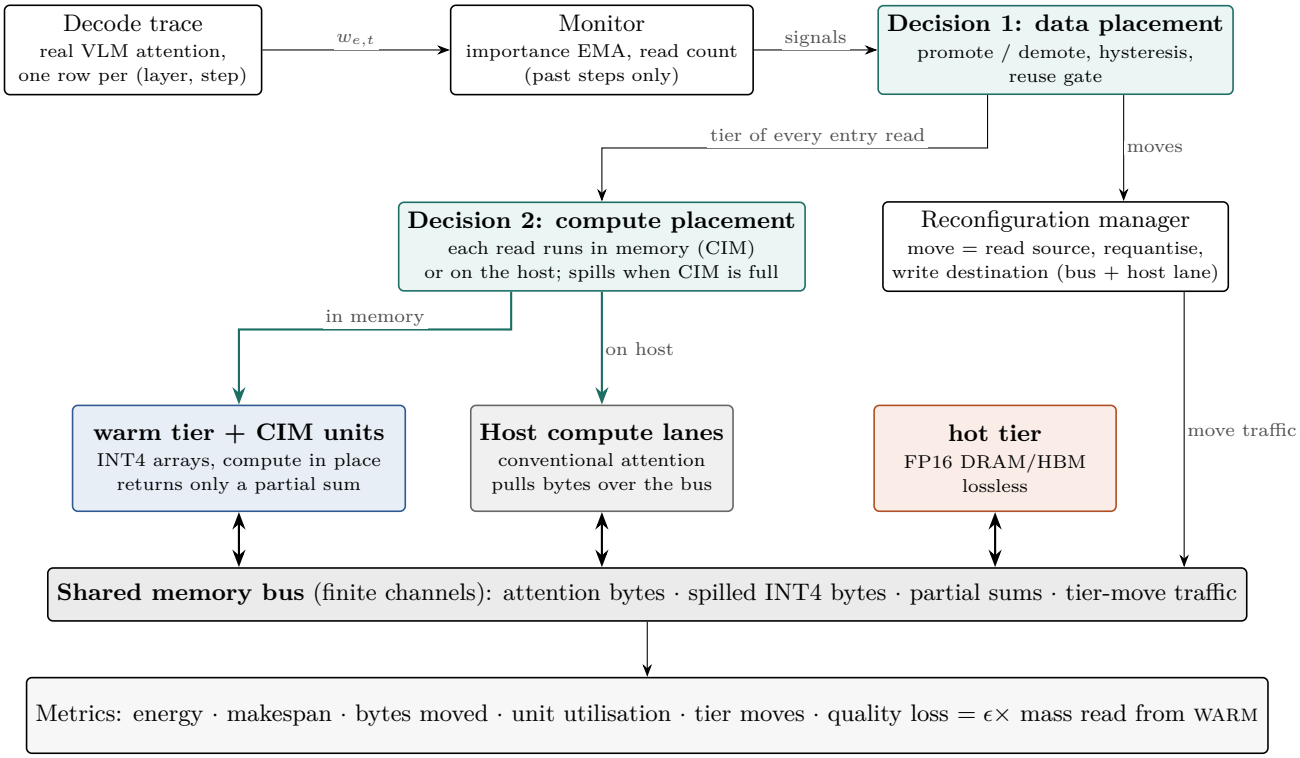


Figure 2: System model. A decode trace supplies, per step, the attention each entry receives. The monitor (past steps only) tracks importance and read counts. Two runtime decisions follow: where each entry's data lives (tier; changed by migrations, which are priced as bus and host-lane traffic) and where each read is computed (in WARM's CIM units, or on the host after pulling the bytes over the bus). All reads, partial sums and migrations share the finite bus; quality is accounted separately from energy and latency.

in the setting we study: given data whose importance and reuse change during execution, and in-memory units that are finite and lossy, *which data should be resident where, which reads should run in memory, and what do the observation and migration cost?*

Memory systems for LLM serving. PagedAttention [11] manages KV memory in fixed blocks. Our contribution is orthogonal: a migration-cost model for *moving* entries between a high-fidelity tier and a low-energy lossy tier, a compute-site scheduler, and an evaluation on real attention under three access regimes, with the conditions under which dynamic placement fails.

Positioning. Our access-cost constants are stylised ratios (Table 2), not a specific device, so absolute energies are not claims about hardware. What the study can support are *ratios between policies under one shared cost model* and the sensitivity of those ratios to the assumptions that matter.

3 System Model

3.1 Entities and tiers

A decode session has a prefill of n_p tokens (visual or text) followed by T=64 generated tokens. Following the per-layer view of Sec. 5.3, one *entry* is the KV pair of one (token, layer); a model with ℓ layers has ℓ entries per token. Every decode step, every active entry is read once (dense reads) with attention weight $w_{e,t}$, $\sum_e w_{e,t}$=1.

3.2 Quality proxy

Reading entry e from WARM perturbs its key and value by INT4 quantization. We charge a quality loss of $w_{e,t} \cdot \epsilon$, so the per-step quality loss is ϵ times the attention mass served from WARM. The simulator's

Table 2: Tier cost model (stylised, one shared set of ratios for all policies). The per-access cost is $c_0 + c_1 \cdot$ bytes at the tier's own precision.

Tier	Precision	Base lat. (ns)	Lat./byte (ns)	Base energy (pJ)	Energy/byte (pJ)
HOT	FP16 (2 B/elem)	20	0.010	5.0	8.0
WARM	INT4 (0.5 B/elem)	30	0.015	0.5	0.8

WARM is 10× cheaper per byte in energy, 1.5× slower per access, and stores 4× fewer bytes. Quality weight of a WARM read: INT4 relative error ϵ.

default is ϵ=0.05, an assumption made before any measurement. We later measured ϵ on real SmolVLM KV tensors (per-token asymmetric min-max INT4, groups of 32): relative L_2 error 0.102 (K) and 0.081 (V), about 0.09 on average and $\sim 1.8\times$ our default; visual and text tokens differ by under 1%. Because loss is linear in ϵ, *every quality ratio between policies is unchanged*, and only absolute quality values scale. Section 5.7 tests whether this proxy actually predicts the real model's behaviour.

3.3 Importance monitor and policy

The monitor keeps an exponential moving average (EMA, decay 0.9) of the attention each entry receives, updated at the end of every step from that step's weights only. The *dynamic hybrid* policy starts each entry at its modality prior (visual → WARM, text → HOT) and then applies a hysteresis rule: promote WARM→HOT when importance exceeds $\theta_\uparrow$ and demote HOT→WARM when it falls below $\theta_\downarrow < \theta_\uparrow$. A warm-up of two EMA updates prevents moves driven by the zero-initialised EMA. Thresholds are expressed in multiples of *uniform attention* $1/n_p$ (so a threshold of 8× means "eight times what a uniform cache would give each entry"), which makes them comparable across traces of different length.

3.4 Reconfiguration and its linearity

A tier change costs m times one HOT access, in both latency and energy (m=10 by default). Policy decisions depend on the importance EMA and never on cost, so a run's totals decompose exactly as

$$E(m) = E_{\text{access}} + m\, E^{(1\times)}_{\text{reconf}}, \tag{1}$$

which is verified by a unit test. We therefore simulate once at m=1 and obtain any other cost as an exact post-hoc sum. This is what makes the cost sweeps in Sec. 5.5 cheap and exact (thresholds are, however, *re-selected* at each cost).

3.5 Baselines and metrics

Flat: everything in HOT. **Static modality**: visual in WARM, text in HOT, never moved; this is the strong baseline throughout and all results are percentages *relative to it*, per trace. **Static layer/role profile**: groups of (modality or role) × layer placed in HOT when their mean importance exceeds a threshold, fixed offline (Sec. 5.4). We report total energy, serialised latency, the quality proxy, and the number of tier moves. **Sparse reads** (Sec. 5.3) read only the top-k% entries by live importance plus entries created in the last step.

3.6 Compute resources, data movement and runtime scheduling (system model)

Experiments E1–E7 use the access-cost model above: the tier implies the compute site, latencies are summed serially and a migration is a lump-sum multiple of a HOT access. That isolates the data-placement question but cannot show contention, data movement or a choice of compute site. E8–E9 therefore use a second model (`system.py`; the first simulator is untouched so E1–E7 stay reproducible) in which resources are explicit and shared (Fig. 2, Tab. 3).

Regression anchor. With one global server, no compute time, no partial-sum traffic and the lump-sum migration cost, the model reproduces the first simulator's energy, latency, quality and move counts to 10^{-9} relative error for the flat, static and dynamic policies (unit test). Everything new is therefore an addition, not a re-parameterisation.

Table 3: System model parameters (defaults used in E8–E9 unless a sweep says otherwise). Tier access costs are those of Tab. 2.

Element	Model
Resources	8 memory-bus channels, 8 host compute lanes, 16 CIM units (swept 2–64 in E8). Work is issued as packets of 64 entries; a packet occupies one unit of a resource at a time and queues FIFO.
HOT read (host)	occupies a bus channel for the HOT access latency (movement), then a host lane for 2 ns/entry (MACs). Energy: HOT access energy.
WARM read in memory	occupies a CIM unit for the WARM access latency; returns a 32 B partial sum per packet over the bus. Energy: WARM access energy plus the partial-sum transfer.
WARM read on host ("spill")	pulls the INT4 bytes over the bus at the HOT-path cost, then dequantises (1 ns, 10 pJ/entry) and computes on a host lane. The data stay INT4, so the quality loss is the same.
Scheduler	`tier`: WARM reads run in memory; `host`: WARM is compressed storage only; `adaptive`: each WARM packet goes where the estimated finish time (from the work already assigned in this step) is earlier.
Tier move (migration)	reads the source tier, requantises and updates metadata (14 ns on a host lane, 60 pJ) and writes the destination tier, all over the bus. This bottom-up cost is $\approx 1.05\times$ one HOT access, versus the $10\times$ assumed in E1–E7; `reconfig_scale` scales it.
Step semantics	Reads of a step are issued together and the step ends at the last packet (a barrier); moves are then issued and also complete before the next step. Energy is additive; latency is the makespan.

Access regimes. The attention weights are always the real ones; a regime says which entries are *read*. *Reusable*: all active entries every step (as in all earlier experiments). *Streaming*: the prefill is consumed once, in order, $\lceil n_p^{\text{tok}}/T \rceil$ tokens per step, plus the newest token, so every entry is read once. *Random*: a uniformly random 3%, 10% or 30% of entries per step, redrawn each step, plus the newest token. Weight on unread entries is reported as skipped mass (identical for all placement policies, so comparisons within a regime are fair). The monitor sees only what is read: an unread entry's importance is not updated. These regimes are access-pattern *overlays* on real attention, not separate real workloads, which we list as a limitation.

Reuse-aware placement. The runtime counts how often an entry was read (reuse = reads/age). A migration of cost R that saves G per read repays only if the entry is read at least R/G more times; R/G is computed from the cost model (1.08 reads at the bottom-up cost, 10.8 at $10\times$ that). The *reuse-aware* policy is the dynamic policy with moves suppressed unless reuse $\times$ 32 steps $\geq R/G$. With dense reads reuse = 1 and it is identical to the dynamic policy (unit test).

4 Evaluation Protocol

Because the policy has free thresholds, the main risk is fitting them to the test set. We adopted three rules and kept them across milestones.

1. **Selection rule.** Thresholds $(\theta_\uparrow, \theta_\downarrow)$ are the pair with the lowest mean energy among those whose mean quality loss is no worse than static modality's on the *training* traces (fallback: lowest quality loss).
2. **Held-out evaluation.** Milestone 5–6: fit on two traces, test on the third. Milestone 7–8: *leave-one-trace-out* over all 42 traces. Results are held-out means.
3. **A fixed pass criterion for the headline claim** (written before the 42-image runs; for E8–E9 the hypotheses H8a–H9e and their criteria were written in the lab notes after the runs were launched but before any result was inspected, Tab. 8): mean held-out energy saving $\geq 10\%$; mean quality-loss change $\leq +2\%$; paired-bootstrap 95% CI of the energy saving excludes 0; the sign holds in each of the three image kinds. Anything weaker is reported as "not established".

Deviations from pre-set plans are stated where they occur: the choice of damping rule (Sec. 5.1), the change of thresholds to multiples of uniform attention after the first real-data failure (Sec. 5.2), the widening of the threshold grid after every fold chose the grid corner (Sec. 5.5), and the minimum-age variant of the reuse gate, added after H9c failed and labelled post hoc (Sec. 5.9). E8–E9 do not re-tune thresholds: the

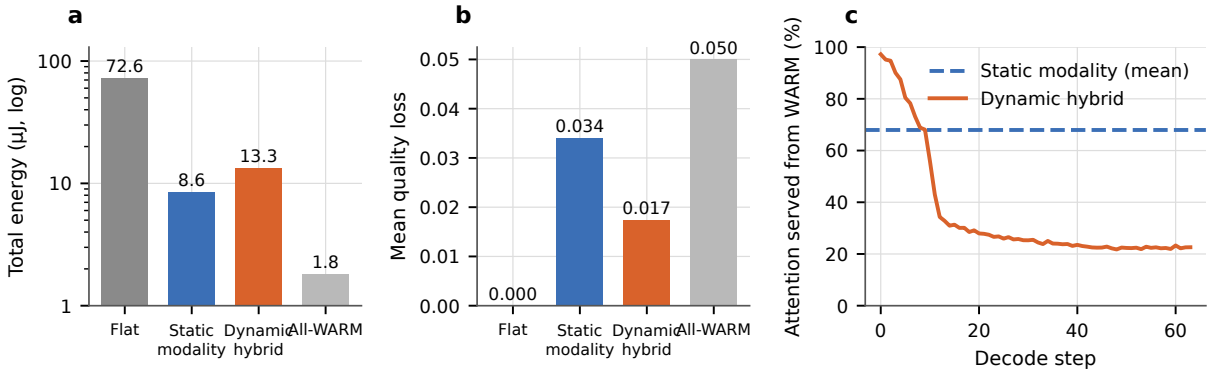


Figure 3: Synthetic trace, default configuration, 10× reconfiguration cost. (a, b) The dynamic policy sits between static modality and flat on energy and halves the quality loss. (c) It does so by promoting the salient visual tokens in the first ∼12 steps, cutting the attention mass served from WARM from ∼97% to ∼36% by step 12 and ∼25% by the end.

E5 choice ($8\times$ /$0.4\times$) is used throughout.

Data. Three locally generated images (chart, scene, noise+label) for Milestones 5–6; 42 images for Milestone 7–8 (24 COCO val2017 photos [17], 10 matplotlib charts, 8 rendered text documents), each with a task prompt. Models: SmolVLM-256M-Instruct (primary) and SmolVLM-500M-Instruct [16]. Per image, we greedy-decode 64 tokens and record, at every step, the head-averaged attention row of every layer (30 layers for 256M). The generated text was checked to confirm the model uses the image.

5 Experiments, in the Order They Built on One Another

The first seven experiments (E1–E7) use the access-cost model of Sec. 3 (the tier implies the compute site; latency is serialised) and ask where *data* should live. E8–E9 use the system model of Sec. 3.6 and ask where *computation* should run and how the answer depends on data reuse.

5.1 E1 — Synthetic traces: is dynamic placement worth its reconfiguration cost?

Question. Does a hysteretic promote/demote policy beat static modality tiering once moves are paid for? **Setup.** A synthetic trace (20 text + 500 visual tokens, 64 steps, a salient 8% of visual tokens, recency bias) reproduces the literature's pattern that visual tokens are on average less important but a salient minority persists.

Step 1: an energy-and-latency-only comparison is degenerate. A trivial *all-WARM* policy beat every dynamic configuration by −78.6% energy at +2.6% latency. With no cost for INT4 error, HOT has no reason to exist. We added the quality term of Sec. 3.2.

Step 2: the policy trades energy for quality. With quality included, the first dynamic policy *lost* to static modality (+67% energy): recency-driven importance spikes at the prefill/decode boundary promoted entries that fell back below the demote threshold within ∼20 steps, so entries round-tripped and each round trip cost 10× a HOT access. Adding a 2-step warm-up (selected over alternatives by quality gain per unit of energy, a post-hoc choice we record as such) cut energy overhead from +64.8% to +55.8% and improved quality from −46.1% to −49.3%; a longer dwell time only batched all moves into one burst.

Finding. On the synthetic trace, dynamic placement buys a large *quality* gain (−48.8% quality loss) for +55% energy over static modality (Fig. 3), a far better rate than interpolating between static and flat (flat: −100% quality loss for +748% energy). **Limitation that drives E2.** This depends on 68% of attention mass landing on WARM-resident salient visual tokens, an assumption of the generator. Whether real VLMs behave this way was untested.

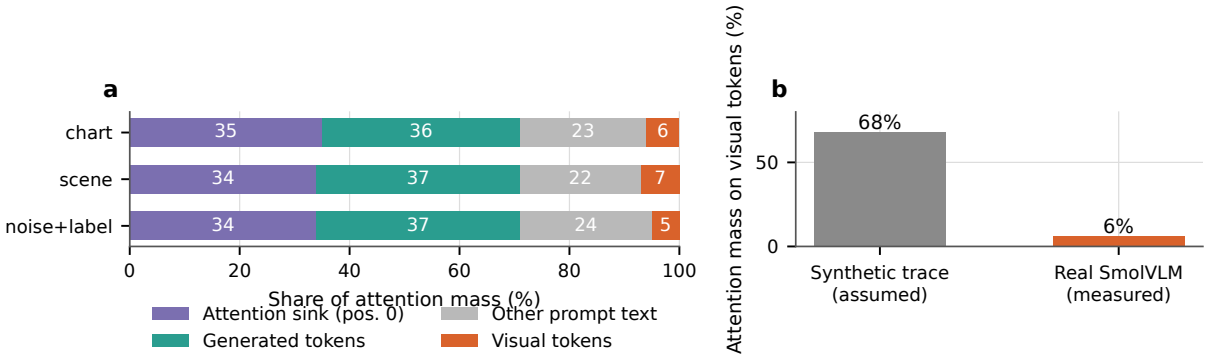


Figure 4: (a) Where real decode attention goes (SmolVLM-256M, three images; layer- and head-averaged). (b) The synthetic trace badly overstated how much attention the WARM tier would serve.

Table 4: E3: held-out trace (fit on the other two), dynamic vs static modality, 10× reconfiguration cost. "Layer-avg" is E2's single entry per token.

Entries / reads	Flat/static energy	Chosen $\theta_\uparrow/\theta_\downarrow$	Energy	Quality loss
Layer-avg, dense	9.3×	2.0/0.4	−14.4%	+11.9%
Layer-avg, top-10%	1.6×	8.0/0.02	+0.2% (no moves)	0%
Per-layer, dense	9.3×	4.0/0.4	**−20.0%**	+0.8%
Per-layer, top-10%	1.9–2.0×	8.0/0.2	+3.7%	−0.2%

5.2 E2 — Real attention: does the synthetic result replicate?

Question. Replace the generator by SmolVLM-256M's measured decode attention (3 images, layer- and head-averaged, one entry per token). **Finding: it does not replicate.** Visual tokens receive only ∼6% of the attention mass (5–7% across the three images), versus 68% served from WARM in the synthetic trace. A single attention sink at position 0 takes ∼35%, and recently generated tokens another ∼36% (Fig. 4a). The visual attention that exists is heavy-tailed and persistent as assumed (the top 8% of visual tokens hold 61–70% of visual mass, with a 74–83% overlap between the first and last step), but the stakes are small: static modality already sits near the quality floor (quality loss ≈0.003), so flat→static is −88 to −90% energy for almost no quality.

A second failure: synthetic thresholds do not transfer. Uniform attention is ≈0.0011 and even the 99th-percentile visual EMA (≈0.0008) is below the synthetic promote threshold (0.006), so nothing visual is ever promoted; the policy then only demotes text, with −23 to −27% energy but +37 to +42% quality loss. Re-expressing thresholds in multiples of uniform attention (a deviation we record) and following the protocol of Sec. 4 (fit on two traces, test on the third) gave thresholds $2.0\times/0.4\times$ and, on the held-out trace, −14.4% energy but +11.9% quality loss: the no-worse-quality criterion **failed**. **Conclusion of E2:** the M4 trade-off was a property of the synthetic trace. We kept the negative result and asked what the layer- and head-averaging was hiding. A per-layer look at one trace shows visual attention mass varying from ∼3% to 39% across layers (layer 0: 39%) and the sink being ≈0% in layers 0–1 but 40–78% in layers 12–30, so one averaged entry per token plausibly hid the structure a policy could use.

5.3 E3 — Per-(token, layer) entries, sparse reads, and measured INT4 error

Changes. (i) One entry per (token, layer), with each layer's attention renormalised so the step's total mass stays 1; (ii) a *sparse-read* model (top-k% entries by live importance plus the newest entries; skipped mass is reported and costs nothing; the importance estimate is an upper bound, oracle and free); (iii) a measured INT4 error (Sec. 3.2). Same protocol as E2.

Finding: the first real-data win. With per-layer entries and dense reads the dynamic policy saves ∼20% energy on all three traces (−20.6/−19.4/−20.0%) at roughly neutral quality (−2.0/−10.3/+0.8%), using 1.2–1.4k tier moves per session (Tab. 4). The held-out quality (+0.8%) technically misses the strict "no worse than static" rule, which we state rather than round. Two boundary results appear: sparse reads shrink the headroom from ∼ 9.3× to ∼ 1.6–2× and the dynamic policy either has nothing to do (layer-average) or loses (per-layer, +3.7 to +7.5%): with sparse reads HOT text entries are rarely read, so

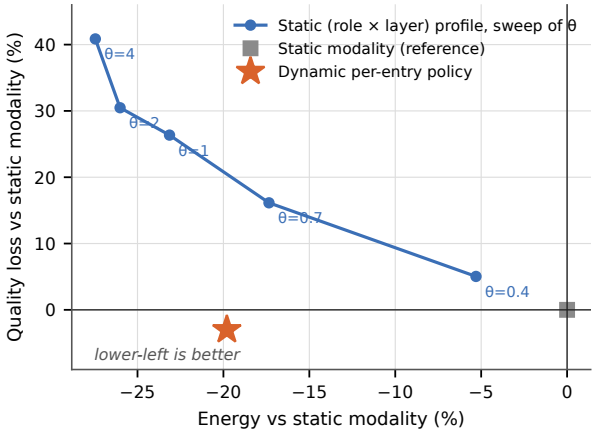


Figure 5: E4: the static (role × layer) profile traces a trade-off curve (dense reads, mean of three held-out folds) and never reaches the dynamic policy. At matched energy ($\approx -20\%$) the static profile loses roughly 16–26% in quality while the dynamic policy gains 3%.

Table 5: E5, dense reads, 10× reconfiguration cost, leave-one-trace-out over 42 images. Percentages are vs static modality, per trace, then averaged; brackets are paired-bootstrap 95% CIs. Quality loss < 0 is better.

Setting	Energy	Quality loss	Selected $\theta_\uparrow/\theta_\downarrow$	Passes criterion
256M, leave-one-out	-22.3% $[-22.8, -21.9]$	-2.5% $[-4.0, -1.1]$	8/0.4 (42/42 folds)	yes
500M, leave-one-out	-23.4% $[-24.1, -22.6]$	-1.8% $[-4.6, +0.9]$	4/0.8 (42/42)	yes
256M thresholds → 500M	-23.6% $[-24.1, -23.1]$	$+0.4\%$ $[-1.4, +2.1]$	8/0.4 (no refit)	yes
256M, top-10% reads	$+5.7\%$ $[+5.4, +5.9]$	-0.7%	32/0.02	**no**
500M, top-10% reads	$+5.7\%$ $[+5.4, +6.0]$	-0.6%	32/0.02	**no**

demoting them saves little while the moves still cost.

What this left open. (a) An explanation: our working hypothesis was a *layer effect*. (b) $n = 3$ images, one model, one held-out trace.

5.4 E4 — Is the per-layer win merely a layer prior? A falsification test

Hypothesis (stated before testing). If the win is a layer effect, a *static* profile, a group importance map by (modality, layer) or (role, layer) with role $\in$ {sink, prompt text, generated, visual} built offline and with zero moves, should capture it. **Protocol.** Leave-one-trace-out profile and threshold θ (eight values from 0.05 to 4.0), plus an in-sample ceiling.

Finding: hypothesis falsified. At matched energy, the static profile has clearly worse quality than the dynamic policy (Fig. 5). The (modality, layer) grouping is degenerate (for $\theta \geq 0.4$ it selects exactly the text groups, i.e. static modality), and the in-sample ceiling ($+27.2\%$ energy, -18.4% quality) nearly equals the cross-validated result ($+27.7\%$, -18.6%), so this is neither under-fitting nor leakage: layer- and role-level information is too coarse. **Correction.** We had written that the saving came from "demoting text entries in layers where text is unimportant". The arithmetic stands (text entries are $\sim$45% of static energy and the saving is about half of that) but the *layer* part was wrong: no text layer group is unimportant on average. The saving comes from variation among individual text entries, i.e. which prompt tokens matter in this prompt, which only a per-entry, online signal sees. **What this left open:** everything rests on three images and one model.

5.5 E5 — Scale-up: 42 images, three image kinds, two model sizes

Pre-specified claims (fixed before running; Sec. 4): (a) the dense-read win holds under leave-one-trace-out; (b) its dependence on the reconfiguration cost; (c) its dependence on sparse read fraction; (d) transfer to a larger model (256M thresholds applied unchanged to 500M). Min-dwell variants were an additional check on move count.

(a) The win is robust for dense reads. The saving is negative on every one of the 42 traces. By image kind (256M): charts -22.8%, documents -20.2%, natural photos -22.9%. Quality is *neutral, not improved*: the per-trace rate of "energy down and quality no worse" is 76% on 256M and 55% on 500M, and the 500M quality interval includes zero. The ladder of Fig. 6 shows how the estimate moved from E2

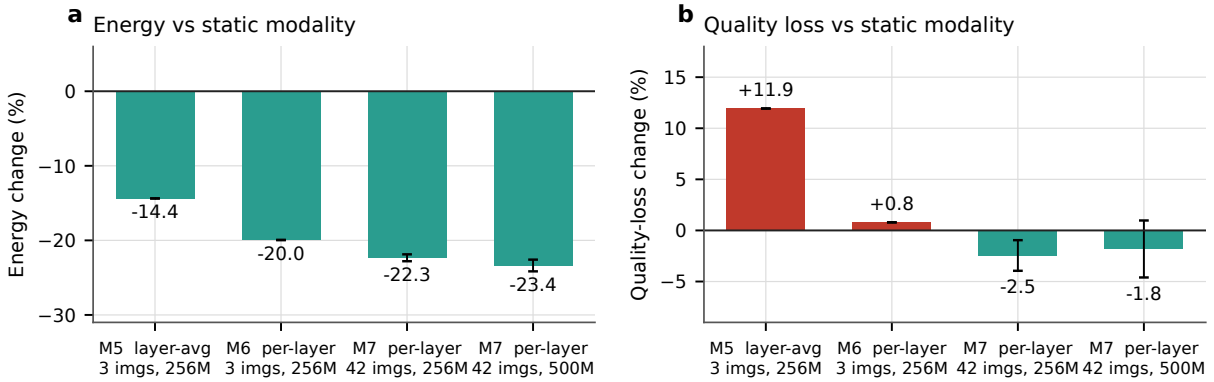


Figure 6: Held-out effect of the dynamic policy across the stages of the study. The first bar is E2 (single entry per token), the second E3 (per layer); the last two are E5. Error bars are 95% CIs (not available for the 3-trace stages).

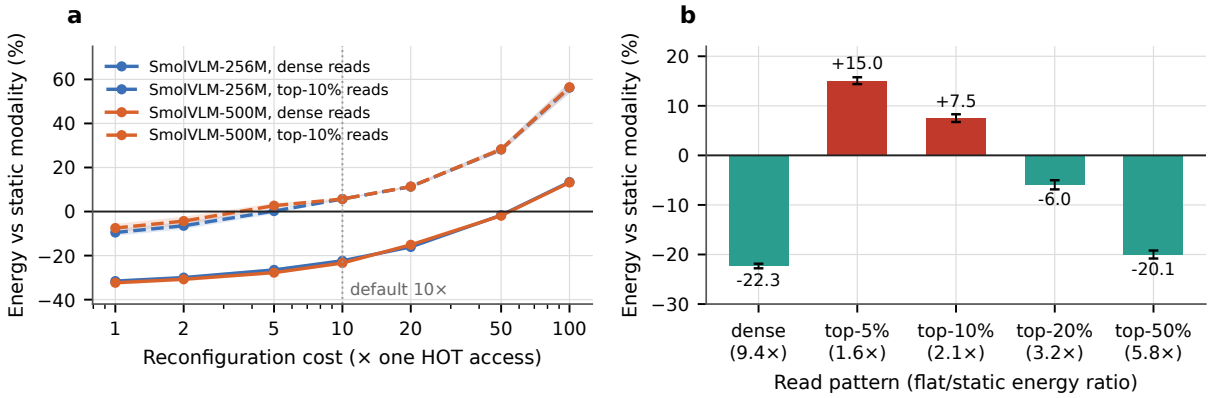


Figure 7: (a) Energy saving vs reconfiguration cost (thresholds re-selected per cost; bands are 95% CIs); the saving breaks even near 50×. (b) Dense reads are required: the saving shrinks and reverses as reads become sparse (flat/static energy ratio in parentheses).

to here.

Grid edge. The first 256M pass selected the grid corner (promote 8×, demote 0.4×) in all 42 folds, which would make leave-one-out meaningless and suggested the optimum might lie outside the grid. We widened the grid (promote up to 32×, demote up to 1.6×) after seeing this and re-ran (first-pass results are kept). The 10× dense result *did not change* (still −22.3%, 8/0.4 in all folds): the corner was a real optimum and not a search limit.

(b) Reconfiguration cost. Re-selecting thresholds at each cost, the saving is −31.6% at 1×, −26.5% at 5×, −16.1% at 20×, −1.7% at 50× and +13.4% at 100× on 256M (500M: −32.3, −27.7, −15.1, −1.9, +13.2). The break-even is about 50× on both models, leaving a factor of about five of margin at the default 10× (Fig. 7a). **(c) Sparse reads.** With top-10% reads the dynamic policy *loses* (+5.7% energy on both models, every trace worse). The 256M sweep over read fraction gives −20.1% (top-50%), −6.0% (top-20%), +7.5% (top-10%) and +15.0% (top-5%) (Fig. 7b): the advantage needs dense or nearly dense reads. The chosen thresholds at 10% (32/0.02) sit at a new grid corner, so we cannot exclude a modest shift with a wider grid, but not a change of sign of this magnitude. **(d) Transfer.** 256M thresholds applied unchanged to 500M give −23.6% energy at +0.4% quality, essentially the same as 500M's own cross-validated optimum.

Mechanism. Ablating the policy at the selected 256M thresholds shows where the saving comes from (Fig. 8): demote-only gives −24.2% energy but *worsens* quality by +6.8%; promote-only costs +1.8% energy and improves quality by −7.6%; the full policy gives −22.3% energy and −0.8% quality. **Demotion earns the energy; promotion buys back the quality it would otherwise give up.** 87% of moves (1008 of 1156 per session) are on prompt-text entries, consistent with E4. A min-dwell requirement only trades saving for fewer moves (0/5/20/40 steps: −22.3/ − 19.8/ − 13.8/ − 5.9%), so the default no-dwell policy is preferred.

What E5 left open. Two objections that no amount of further scaling answers: (1) the policy consumed the *true* attention of every entry for free; (2) "quality" is an attention-weighted error proxy, not

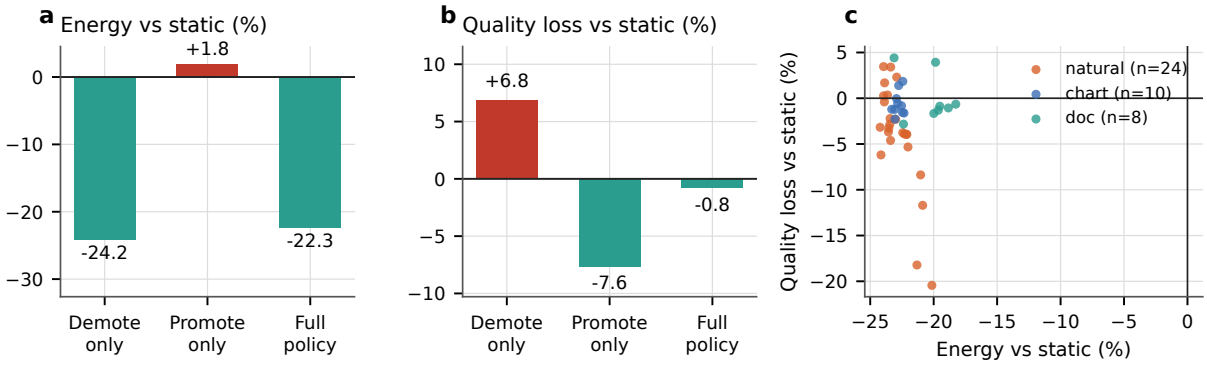


Figure 8: Mechanism ablation on 256M at the selected thresholds (a, b), and the per-trace outcome of the full policy by image kind (c): all 42 points have negative energy change; quality scatters around zero.

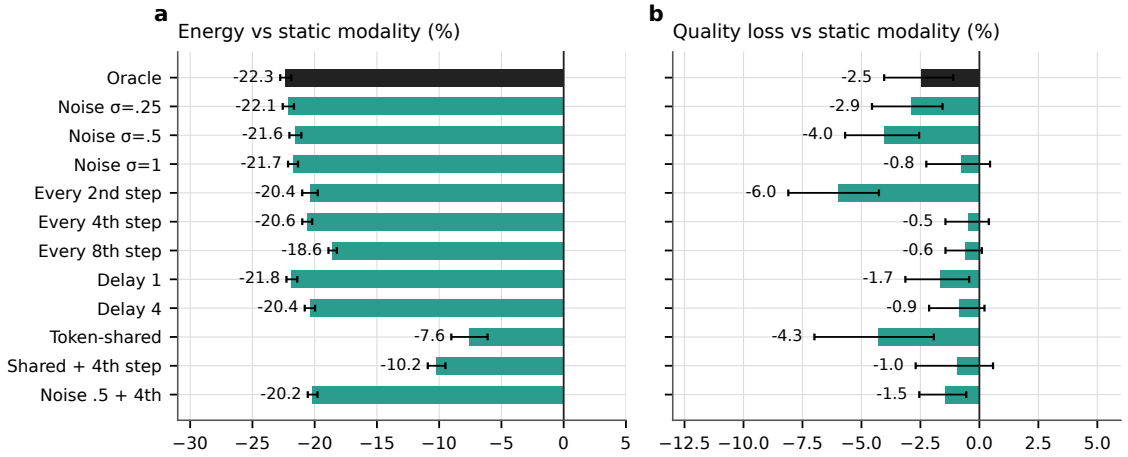


Figure 9: E6: held-out effect of the dynamic policy under imperfect importance signals (256M, 42 traces, dense reads, 10×; bars are leave-one-out means with bootstrap 95% CIs; black = oracle). Per-entry signals can be noisy, sparse in time or stale and keep ≥18.6% of the saving; a single score per token does not.

a measurement of what the model says. The next two experiments address them.

5.6 E6 — Objection 1: the policy used the true attention for free

Question. E3–E5 fed the importance monitor the exact attention weight of every (token, layer) entry at every step. A real system must compute, approximate or sample this signal. How much of the saving survives an imperfect one? **Setup.** An *estimator* sits between the true weights and the monitor (Fig. 2); quality and access energy are still charged with the *true* weights, so the policy simply acts on a worse signal. We test seven degradations on the 42 256M traces with dense reads, 10× cost and the same leave-one-out protocol (thresholds re-selected per estimator over a 10-configuration grid; the oracle row reproduces E5):

- *Noise* $\sigma \in \{0.25, 0.5, 1\}$: each weight is multiplied by an independent log-normal factor (for σ=1, a factor within $e^{\pm 1}$ for 68% of reads and mean-inflated by 1.65×).
- *Sampled*: the monitor sees attention only every 2nd, 4th or 8th step (EMA decay raised to the same power to keep its time constant).
- *Stale*: the monitor sees the weights from 1 or 4 steps ago.
- *Token-shared*: one score per token, the mean over layers, shared by all of its entries, i.e. what a system obtains if it can afford only one importance value per token rather than one per (token, layer).
- Two combinations (token-shared with 4-step sampling; noise σ=0.5 with 4-step sampling).

Finding 1: the saving is robust to a degraded per-entry signal. Noise up to σ=1 leaves −21.8% energy (oracle −22.3%); observing attention only every 8th step gives −18.6%, a signal delayed by 4 steps gives −20.4%, and noise plus 4-step sampling gives −20.2% (Fig. 9). All of these meet the pre-specified criterion (energy saving ≥ 10%, quality change ≤ +2%, CI excluding 0). Quality does not degrade: it stays

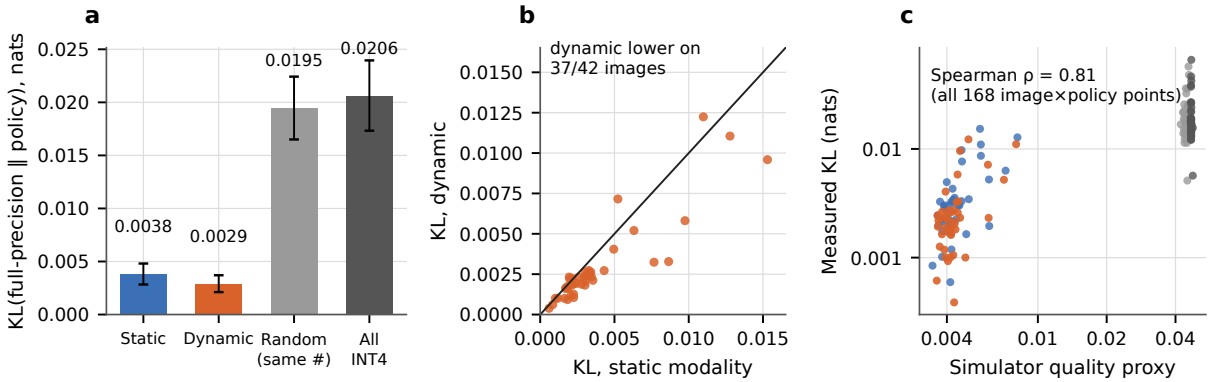


Figure 10: E7: real output change under INT4 placement, 42 images, SmolVLM-256M. (a) Mean KL to the full-precision model (95% CIs). (b) Dynamic vs static per image: points below the diagonal favour the dynamic policy. (c) The simulator's quality proxy against measured KL for all 168 (image, policy) points.

Table 6: E7: measured effect of INT4 placement on the next-token distribution (mean over 42 images). Fraction of entries in INT4 is the mean over steps.

Placement	INT4 entries	KL (nats)	Top-1 agreement	Proxy ϵmass
Static modality	91.8%	0.00382	0.976	0.0045
Dynamic (ours)	94.9%	**0.00291**	0.977	0.0044
Random, same INT4 count	94.9%	0.01946	0.930	0.0445
All INT4	100%	0.02064	0.929	0.0468

at or below the static baseline (between −6.0% and −0.5%), because a noisier or older signal promotes slightly more entries. The number of moves is essentially unchanged (1111–1298 against 1190 for the oracle). **Because the saving equals the energy a system may spend on estimation, the oracle's 22.3% of static access energy is also the upper bound on the estimator's own cost per session**, and a signal sampled every 4th step still leaves ∼20% of headroom.

Finding 2: the signal must stay per (token, layer). Collapsing importance to one score per token cuts the saving to −7.6% [−9.0, −6.1] (with 4-step sampling, −10.2%), below the 10% criterion. At the oracle's thresholds the token-shared signal reaches −18.7% energy but loses +12.8% quality, so the policy cannot find an operating point as good as the per-entry one. This is the same lesson as E4, seen from the signal side: what the policy exploits is variation *between layers of the same token*, not a per-token summary. **Caveat.** All estimators here are free; we charge no energy or latency for producing them, and do not model any specific approximate scorer (e.g. low-precision or sampled key scores). E6 shows how imperfect the signal may be, not what a concrete estimator costs.

5.7 E7 — Objection 2: is the quality proxy real?

Question. Everything so far scores quality with $\epsilon\times$(attention mass served from WARM). Does a policy that the proxy ranks better also change the real model's output less when INT4 is actually applied? **Setup.** For each of the 42 images, we replay each policy's placement on the real SmolVLM-256M. The simulator logs, for every decode step, which (token, layer) entries are in WARM; the model is then teacher-forced on its own full-precision 64-token generation, and at step t the cached K and V of exactly those entries are replaced by their INT4 round trip (per-token asymmetric min-max, groups of 32). We measure the KL divergence $\mathrm{KL}(p_{\text{full}} \,\|\, p_{\text{policy}})$ of the next-token distribution (mean over the 64 steps) and top-1 agreement with the full-precision model. The replay reproduces the saved generation on all 42 images. Four placements are compared: static modality, the dynamic policy (selected thresholds 8/0.4), *random placement with the same number of INT4 entries at every step as the dynamic policy* (a control for *which* entries are demoted), and everything in INT4.

Finding 1: the policy picks the right entries. The dynamic policy places *more* entries in INT4 than static modality (94.9% vs 91.8%) and still changes the output distribution **21.8% less** (per-image mean, 95% CI [15.3, 28.1]%; lower on 37 of 42 images: 22/24 natural, 9/10 chart, 6/8 document). Random placement with the same INT4 count is **9.4**× worse than the dynamic policy ([7.8, 11.1]×; worse on all

42 images) and indistinguishable from putting everything in INT4. The energy saving of E5 is therefore not obtained by demoting entries at random, and the entries the policy keeps in HOT are the ones the real model is sensitive to.

Finding 2: the proxy is a usable but conservative guide. Across all 168 points the proxy and the measured KL are strongly rank-correlated (Spearman ρ=0.81; Pearson on log scale 0.88), mostly because the proxy separates policies that demote important entries from those that do not (Fig. 10c). Within the static/dynamic pair the correlation is weaker (ρ=0.49), the proxy's per-image sign of the dynamic-vs-static difference matches the measured one on 29 of 42 images, and it *understates* the advantage: the proxy predicts −2.5%, the measurement −21.8%. Top-1 agreement does not separate dynamic from static (difference +0.1 points, CI includes 0). **Caveats.** The effects of INT4 on this small model are small in absolute terms (even all-INT4 keeps 93% top-1 agreement over 64 teacher-forced steps), so E7 validates the *ordering and mechanism* rather than a task-accuracy claim, covers one model and the single INT4 scheme measured in Sec. 3.2, and uses KL as the quality measure, not a benchmark score.

5.8 E8 — Objection 3: where should each read run? Computation placement under contention

Question. E1–E7 chose where data live, and the tier implied where a read was computed, with latencies summed serially. A memory-centric design must also decide *where the computation runs* when compute units are finite and share a data path. Does the compute-site decision matter, and does the data-placement result survive contention? **Setup.** The system model of Sec. 3.6, dense (reusable) reads, 42 traces on SmolVLM-256M. Policies: flat, static modality, and the dynamic policy at the E5 thresholds (8 × /0.4×, *not* re-tuned) with the bottom-up migration cost. Schedulers: `tier` (WARM reads run in memory), `host` (WARM is compressed storage; every read runs on the host) and `adaptive`. CIM capacity is swept from 2 to 64 units; 8 bus channels and 8 host lanes are fixed. Hypotheses and pass criteria H8a–H8c were written in the lab notes after the runs were launched but before any result was inspected (apart from one smoke-test trace). Values are means over traces of per-trace ratios to flat (the trace-to-trace s.d. of every ratio is below 0.01 for energy and below 0.04 for makespan).

H8a — contention reverses the serial-latency penalty: confirmed. With 16 CIM units, static modality moves 11.8× fewer bytes over the bus than flat, uses 9.3× less energy and has a *lower* makespan (0.64× flat's). Flat is bus-bound (99% bus utilisation); static modality is CIM-bound (98%). In the serialised model of E1–E7 the same placement looked ∼34% *slower* than flat, because serialisation forbids overlapping the two tiers. Part of what E1–E7 called a latency cost of tiering was an artefact of the cost model.

H8b — the compute-site decision is first-order: confirmed. If WARM reads always run in memory, the verdict depends on CIM capacity: static modality is 5.0×, 2.5× and 1.25× *slower* than flat with 2, 4 and 8 units, and breaks even near 16 (Fig. 11a). The opposite extreme, treating WARM as compressed storage computed on the host, is capacity-independent (0.92× flat's makespan) but gives up most of the benefit: energy 0.319 of flat versus 0.108, and bytes on the bus 0.313 versus 0.085 (Fig. 11b,c). The adaptive scheduler is better on makespan than *both* at every capacity (e.g. 0.42× flat at 16 units, against 0.64× and 0.92×), because it keeps the CIM units and the host lanes busy at once; it spills 83% of WARM packets at 2 units and 6% at 64 (Fig. 11d). The price is energy: at 16 units adaptive uses 1.66× the energy of in-memory compute (0.179 vs 0.108 of flat) for a 33% lower makespan, still far below host-only (0.319). Compute placement is therefore a latency–energy trade-off that the runtime resolves from queue state, not a property of the tier.

H8c — data-placement gains survive contention: confirmed, with a latency cost. At 16 CIM units with in-memory compute, the dynamic policy lowers energy by **27.4%** against static modality (95% CI [27.0, 27.8]; larger than E5's −22.3% because the bottom-up migration costs ≈1.05 rather than 10 HOT accesses) and bytes on the bus by 36%, at quality 2.5% better. It is 4.7% ([4.6, 4.9]) *slower* in makespan, plausibly because it keeps more entries on the host-side path and its 1.1k moves per session compete for the bus (we did not isolate the two). Under the adaptive scheduler the energy advantage over static shrinks to 12.5% at 16 units and to 8.0% at 2: spilled reads cost the same on the host regardless of how WARM came to hold the data, so a scarce CIM tier dilutes what any tiering policy can save.

What E8 left open. All of E1–E8 read every entry every step (or the top-k of it). The task statement's own example has data with different *reuse*: re-read, streamed once, accessed at random. E9 tests whether placement should differ across them.

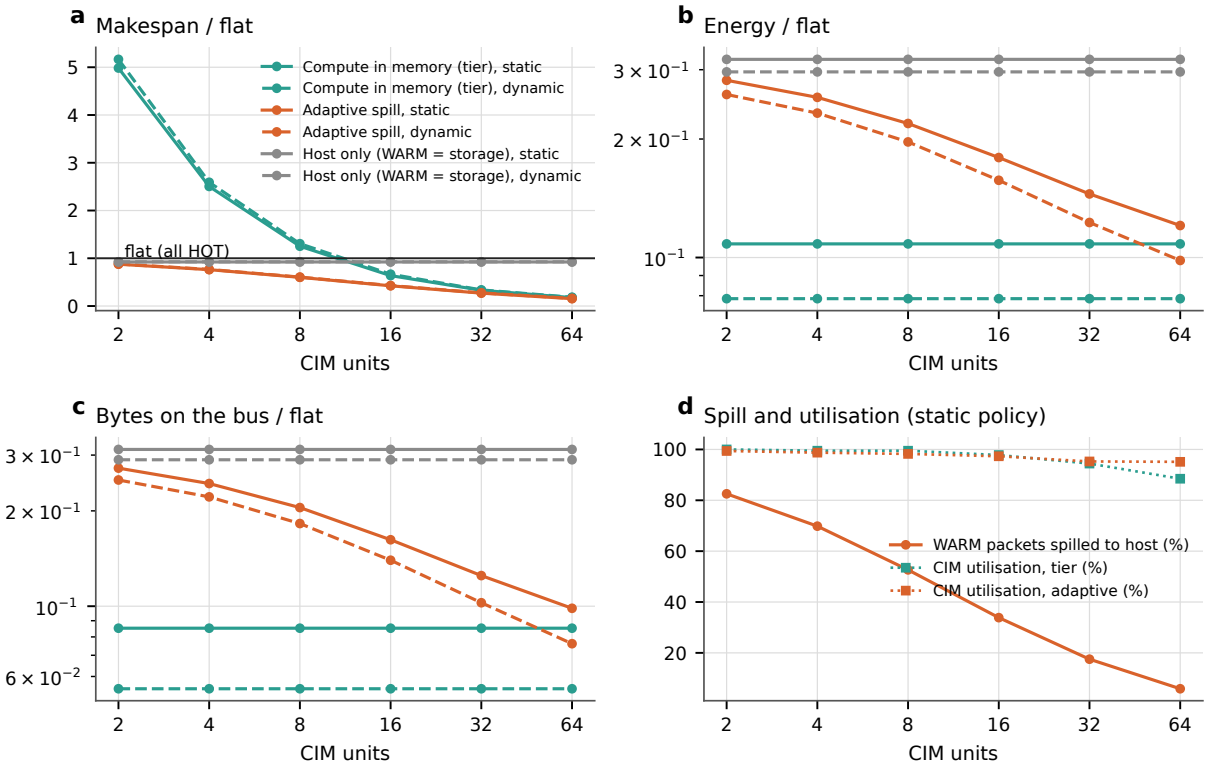


Figure 11: E8: compute-site scheduling vs CIM capacity (dense reads, 42 traces; solid = static modality, dashed = dynamic). (a) Makespan relative to flat: compute-in-memory alone is slower than flat below ∼16 units; host-only is capacity-independent; adaptive spill beats both. (b) Energy and (c) bytes on the bus relative to flat. (d) Fraction of WARM packets the adaptive scheduler spills to the host, and CIM utilisation.

5.9 E9 — Objection 4: should placement differ for reusable, streamed and randomly accessed data?

Question. The same system may process a highly reusable dataset, a streaming dataset and a randomly accessed one. Everything up to E8 read each entry every step, so reuse was uniform. Does the best placement depend on the access regime, and can the runtime adapt without being told the regime? **Setup.** The three regimes of Sec. 3.6 (reusable = dense; random with 30%, 10% and 3% of entries read per step; streaming = each entry read once), 42 traces, 16 CIM units, in-memory compute. Policies: static modality, the dynamic policy ($8\times/0.4\times$, not re-tuned for any regime) and the reuse-aware policy of Sec. 3.6, each at the bottom-up migration cost and at $10\times$ that cost (a stress setting, equal to the cost assumed in E1–E7). Random draws use one seed per trace, so every policy sees the same reads. H9a–H9e were written down after the runs were launched and before any result was inspected.

H9a, reusable data: confirmed. Dynamic placement lowers energy by 27.4% at the bottom-up cost (95% CI $[26.9, 27.8]$) and by 21.6% at $10\times$ (E5's -22.3% in the first simulator), with quality 2.5% better ($[1.1, 4.0]$), 1190 moves per session, and a makespan penalty of 4.7% and 13.1% respectively.

H9b, streaming data: confirmed, trivially. With one read per entry, no entry is observed twice. The monitor's warm-up (Sec. 3.3) therefore never lets a move happen, and the dynamic and reuse-aware policies are identical to static modality in all metrics. The right decision for streamed data is simply the initial placement. The reason is worth stating because it is not a finding about the policy's cleverness: read-once data *cannot repay* a migration, and the warm-up happens to encode that. (In this regime flat is 14% *faster* than static but uses $7.3\times$ the energy: the compute-site trade-off of E8 reappears with few reads.)

H9c, random access: not confirmed as stated. At $10\times$ migration cost the ungated dynamic policy is much worse than static when reads are sparse (+32.1% energy at 10% reads, +81.9% at 3%), because it pays ∼1000 moves per session for entries that are read too rarely to repay them. The gate removes most of the harm by suppressing 91% and 98% of those moves (1064→91 and 645→9) and brings the loss to +2.2% and +1.1%, but our criterion was parity within 1%, and the gate does *not* meet it. With the bottom-up

Table 7: E9: change against static modality (mean over 42 traces; energy and quality loss in %, negative is better). "Gate" is the reuse-aware policy (the post-hoc variant of Sec. 5.9 is in Fig. 12). Flat/static is the energy headroom of WARM in that regime.

	Flat/	Dynamic, bottom-up cost			Dynamic, 10×		+ gate, 10×	
Regime	static	Energy	Quality	Moves	Energy	Quality	Energy	Moves
Reusable (dense)	9.3×	−27.4	−2.5	1190	−21.6	−2.5	−21.6	1190
Random, 30% read	9.1×	−25.1	+3.9	1038	−8.5	+3.9	−5.4	744
Random, 10% read	8.6×	−15.8	+5.7	1064	+32.1	+5.7	+2.2	91
Random, 3% read	7.3×	+2.1	+2.6	645	+81.9	+2.6	+1.1	9
Streaming (read once)	7.3×	0	0	0	0	0	0	0

Table 8: Pre-specified hypotheses of E8–E9 and their outcome.

	Statement (abbreviated)	Outcome
H8a	Tiering moves ≥5× fewer bytes and uses ≥5× less energy than flat, at lower makespan	Confirmed (11.8×, 9.3×, 0.64×)
H8b	Compute site matters: in-memory-only slower than flat at ≤4 CIM units; adaptive within 10% of the better pure strategy, cheaper than host-only	Confirmed (adaptive better than both)
H8c	Dynamic ≥10% lower energy than static under contention, quality within +2%	Confirmed (−27.4%, −2.5%); +4.7% makespan
H9a	Reusable: dynamic ≥10% lower energy, neutral quality	Confirmed (−27.4% / −21.6%)
H9b	Streaming: dynamic ≈ static	Confirmed (identical; no moves)
H9c	Random, 10× cost: ungated dynamic loses; gated within +1% of static	Ungated loses; **gate misses** (+2.2%, +1.1%)
H9d	Best policy differs by regime; gated never > 2% worse than static	Half: differs; **misses** narrowly (+2.2%)
H9e	Break-even migration cost (reported)	≈45× the bottom-up estimate

cost the break-even is only 1.08 reads, the gate never engages, and the dynamic policy saves energy under random access (−15.8% at 10% reads, −25.1% at 30%) *but loses quality* (+5.7% and +3.9% against static; +2.6% at 3%, where it also loses energy). Dynamic placement is therefore not regret-free under random access even when migration is cheap. Its thresholds were fitted to dense reads, and a monitor that only observes entries when they happen to be read is a plausible but untested cause.

H9d, no single policy: half confirmed. The best policy differs by regime: dynamic for reusable data, static for sparse random access (10% and 3%), either for streams. The second part (the reuse-aware policy never exceeds static by more than 2% energy) fails narrowly: +2.2% at 10% reads and 10× cost.

Post-hoc diagnosis of the residual (not part of the protocol). After seeing H9c fail we asked why the gate still lets ∼90 moves through. A look at one trace (82 moves, 79 of them on prefill entries) suggested a noisy read-frequency estimate early in the session: an entry read a few times in its first steps looks highly reused. Requiring a minimum age before the estimate is trusted tests that explanation. With a minimum age of 16 steps the gate removes the residual at 10×: +0.13% energy ($[0.11, 0.15]$) and neutral quality at 10% reads, 0.0% at 3% (no moves), −0.8% at 30%, with 4.5 moves per session at 10%. The price is paid in the reusable regime, where delaying early moves cuts the saving from −21.6% to −14.6% (and −27.4% → −20.0% at the bottom-up cost). The choice matters: ages 4 and 8 leave +2.15% and +1.06%. Because the variant was designed after seeing these test traces, with three ages tried, it is a diagnosis of where the regret comes from, not a tuned headline result.

H9e, migration-cost sensitivity. In the explicit model (dense reads, 14 traces, fixed thresholds) the saving is −28.1% at the bottom-up cost, −25.5% at 5×, −22.3% at 10×, −15.9% at 20× and +3.3% at 50× (CI $[+2.3, +4.7]$), a break-even near 45× the bottom-up estimate, in agreement with E5's ∼50× from a different cost model. Migration traffic is 1%, 8%, 15% and 31% of total energy at 1×, 10×, 20× and 50×, and the makespan penalty grows from +4.6% to +47.6%: with a shared bus, migrations cost time as well as energy.

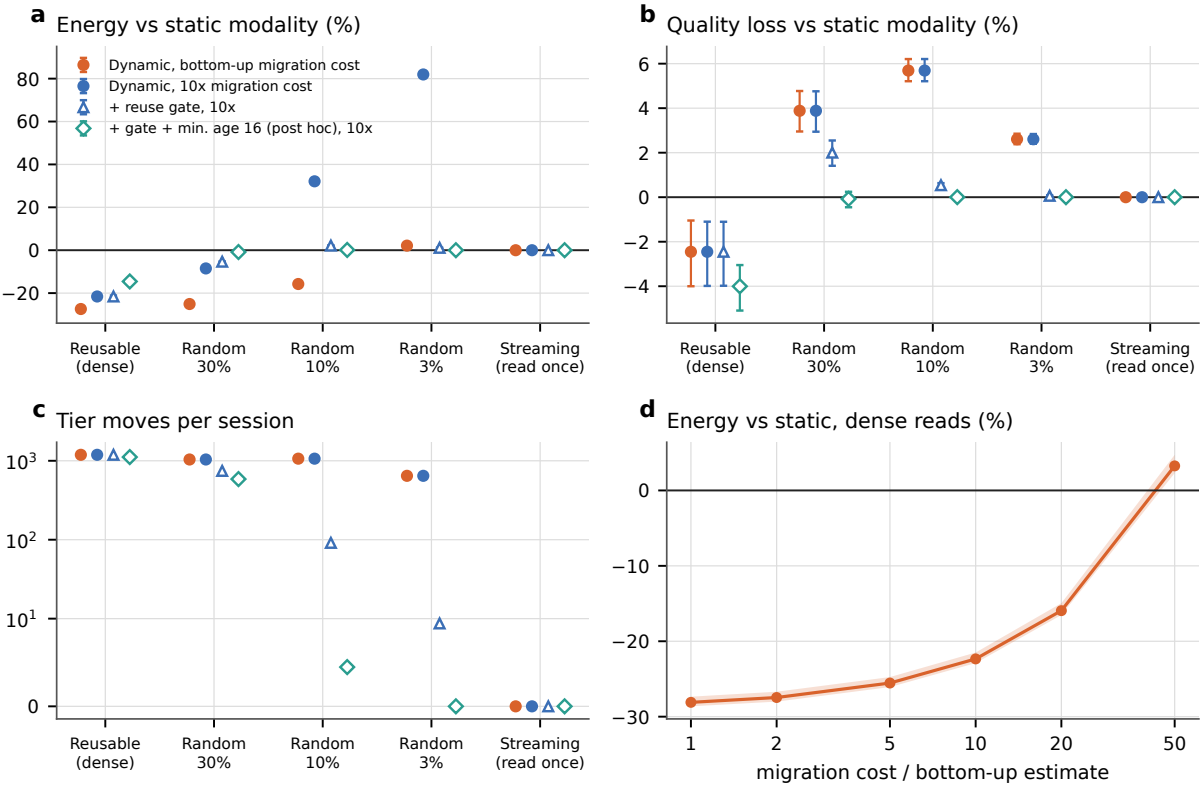


Figure 12: E9: (a) energy and (b) quality loss against static modality, by access regime (95% CIs). (c) Tier moves per session. (d) Energy against migration cost in the dense regime (14 traces; band = 95% CI): break-even near 45× the bottom-up estimate. The green diamonds are the post-hoc variant.

What E8–E9 establish. Computation placement is a first-order decision (a wrong default is 5× slower than no tiering at 2 CIM units); a runtime that watches queue state beats both fixed choices on makespan; and whether dynamic *data* placement pays depends on reuse: strongly for reusable data, not at all for streams, and under random access only if migrations are priced against the reuse the runtime has observed. The reuse-aware policy removes most but, as pre-specified, not all of the regret under random access.

6 Discussion

What is established. For dense per-layer reads, under one stylised but fixed cost model, importance-driven placement beats the strong static-modality baseline by ∼22–23% energy at neutral quality on two model sizes and three image kinds, with thresholds that transfer across sizes and a break-even reconfiguration cost about five times the default. The saving is produced by demoting individual prompt-text entries whose importance is low *in this prompt*, and the promotions that give quality back are few. The two added checks narrow the main objections: the gain does not require an oracle (it survives noisy, stale or sampled importance, provided importance stays per (token, layer)), and it is not an artefact of the quality proxy (when INT4 is applied in the real model, the policy keeps in HOT the entries that matter: its output divergence is 21.8% below static modality's and 9.4× below random placement's). They do not replace hardware measurement, and neither the estimator's own cost nor a concrete approximate scorer is modelled. Under explicit resources (E8) the result holds and is stronger than in the serial model (−27.4% at the bottom-up migration cost, with 11.8× fewer bytes on the bus than flat), but the compute-site decision it rests on is first-order: with scarce CIM units, computing every WARM read in memory is up to 5× slower than not tiering at all, and a scheduler that spills to the host is better than either fixed choice. Whether data placement should be dynamic depends on reuse (E9).

Where it does not apply. (i) Sparse reads: with top-10% reads the policy is a net loss and the HOT/WARM headroom itself shrinks to ∼2×. (ii) Reconfiguration costs above roughly 50× a HOT access.

(iii) Coarse-grained placement: a static layer or role map cannot reproduce the effect. (iv) Read-once (streamed) data: nothing can repay a migration and the right decision is the initial placement. (v) Sparsely and randomly read data: the dynamic policy is not regret-free even at the bottom-up migration cost (quality +3.9 to +5.7% against static at 30% and 10% reads), and at 10× it loses up to +82% energy; a reuse gate removes most but not all of that. These boundaries are as much a part of the finding as the saving.

Why the negative results stayed in. E2 and E4 overturned beliefs we held (that the synthetic trade-off was general; that the win was a layer effect). Keeping them is what makes the later claim credible: the scale-up in E5 was designed to be able to fail, and the criterion was fixed beforehand.

Limitations.

- **Stylised cost model.** WARM is 10× cheaper per byte and 1.5× slower than HOT; the reconfiguration multiplier is swept but these ratios are not. Results are ratios between policies under shared assumptions, not device energies. There is no hardware measurement.
- **Models and data.** Two sizes of one model family, 42 images, 64 decode steps, one prompt per image. Larger models (we planned SmolVLM-2.2B and dropped it), other families and long-form generation are untested. Attention is head-averaged per layer; per-head entries are not modelled.
- **Two tiers, simple resources.** No capacity pressure on HOT. Contention exists only in E8–E9 (E1–E7 serialise latency) and is a small model: fixed counts of bus channels, host lanes and CIM units (only CIM is swept), FIFO queues, a per-step barrier, migrations that do not overlap reads, and a greedy scheduler with a work-conserving finish-time estimate. Bank conflicts, row-buffer behaviour, refresh and a real CIM array's constraints are not modelled.
- **Access regimes are overlays.** The streaming and random regimes re-read the same real attention rows under a different read pattern; they are not separate real workloads. One query per image, decode only, no batching, no prefill cost.
- **Scope of the two added checks.** E6 charges nothing for computing the importance signal. E7 covers one model, one INT4 scheme (the one measured in Sec. 3.2) and measures distributional change (KL over 64 teacher-forced steps), not benchmark accuracy; INT4 effects on this small model are small in absolute terms.
- **Post-hoc choices** (logged above): the damping rule in E1, the threshold normalisation in E2, the grid widening in E5, the minimum-age variant in E9. The migration cost in E8–E9 is derived bottom-up from the same stylised tier table, so it is internally consistent but not a hardware measurement.

7 Conclusion

We asked where the computation on a multimodal KV cache should run, and answered with a chain of nine experiments in which every step confirmed or overturned the last. Dynamic, importance-driven placement of data between a conventional tier and a compute-in-memory tier pays off by ∼22–27% energy at neutral quality when data are reused, for as long as migration costs less than about 45–50× a conventional access, and it survives imperfect importance estimates and real INT4 error in the model. Under explicit shared resources the decision of *where each read is computed* is as important as where data live: a wrong default is up to 5× slower than no tiering, and a scheduler that watches queues and spills to the host is better than either fixed choice. Streamed data need no dynamic placement, and sparsely or randomly read data make dynamic placement a liability unless migrations are priced against observed reuse, which our reuse gate does only in part. The study also leaves a usable method: fix hypotheses and pass criteria before looking, report held-out numbers with intervals, say plainly which pre-specified criteria failed, and keep the experiments that did not work.

Reproducibility. All code, traces, per-simulation CSVs and this paper's figure scripts are in the repository. `experiments/run_m7.py`, `analyze_m7.py`, `run_m7_estimators.py`, `validate_real_output.py` and `paper/make_figures.py` regenerate Sections 5.5–5.7, and `src/kvtier/system.py`, `experiments/run_m9.py` and `analyze_m9.py` (E8–E9; 3,402 simulations, about 25 minutes on 7 cores); the lab notebook `NOTES.md` records every deviation chronologically.

References

[1] Tu, D., et al.: VL-Cache: Sparsity and modality-aware KV cache compression for vision-language model inference acceleration. arXiv (2024).

[2] He, E., et al.: ZipVL: Efficient large vision-language models with dynamic token sparsification. arXiv (2024).

[3] Huang, K., et al.: AirCache: Activating inter-modal relevancy KV cache compression for efficient large vision-language model inference. arXiv (2025).

[4] Yang, C., et al.: TopV: Compatible token pruning with inference time optimization for fast and low-memory multimodal vision language model. CVPR (2025).

[5] Chen, L., et al.: An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. ECCV (2024).

[6] Zhang, Z., et al.: H2O: Heavy-hitter oracle for efficient generative inference of large language models. NeurIPS (2023).

[7] Li, Y., et al.: SnapKV: LLM knows what you are looking for before generation. NeurIPS (2024).

[8] Xiao, G., et al.: Efficient streaming language models with attention sinks. ICLR (2024).

[9] Liu, Z., et al.: KIVI: A tuning-free asymmetric 2bit quantization for KV cache. ICML (2024).

[10] Tang, J., et al.: Quest: Query-aware sparsity for efficient long-context LLM inference. ICML (2024).

[11] Kwon, W., et al.: Efficient memory management for large language model serving with PagedAttention. SOSP (2023).

[12] Mutlu, O., Ghose, S., Gómez-Luna, J., Ausavarungnirun, R.: Processing data where it makes sense: Enabling in-memory computation. Microprocessors and Microsystems 67 (2019).

[13] Lee, S., et al.: Hardware architecture and software stack for PIM based on commercial DRAM technology. ISCA (2021).

[14] Park, J., et al.: AttAcc! Unleashing the power of PIM for batched transformer-based generative model inference. ASPLOS (2024).

[15] Heo, G., et al.: NeuPIMs: NPU-PIM heterogeneous acceleration for batched LLM inferencing. ASPLOS (2024).

[16] Marafioti, A., et al.: SmolVLM: Redefining small and efficient multimodal models. arXiv (2025).

[17] Lin, T.-Y., et al.: Microsoft COCO: Common objects in context. ECCV (2014).

A Glossary

KV cache Per-layer keys and values of every token already processed, re-read at every decoding step.

Entry The KV pair of one (token, layer); the unit that is placed.

hot / warm Conventional FP16 DRAM/HBM tier with host compute / INT4 compute-in-memory tier (cheaper per byte, slower, lossy).

CIM Compute-in-memory: arithmetic done inside the memory array, so that only a partial result crosses the bus.

Static modality Baseline: visual entries in WARM, text entries in HOT, never moved.

Dynamic policy Promotes/demotes entries on a live importance EMA with hysteresis, paying for every move.

Migration A tier move: read source tier, requantise, write destination tier.

Spill A WARM read computed on the host instead of in memory.

Makespan Simulated time of the whole session with resource contention (E8–E9); E1–E7 report serial latency.

Reuse rate Fraction of steps in which an entry was read; the gate in E9 extrapolates it to decide whether a migration will be repaid.

KL $\mathrm{KL}(p_{\text{full}} \,\|\, p_{\text{policy}})$ of the next-token distribution, used in E7.

B Pre-specified hypotheses of E8–E9

Written in the lab notes (`NOTES.md`, section M9) after the runs were launched and before any result was inspected, apart from one smoke-test trace. Thresholds are the E5 choice ($8\times$ /$0.4\times$) throughout.

H8a Static modality vs flat moves $\geq 5\times$ fewer bus bytes and uses $\geq 5\times$ less energy, and its makespan is *lower* than flat's.

H8b At ≤ 4 CIM units, always computing in memory makes static modality *slower than flat*; the adaptive scheduler is within 10% of the better of {tier, host} at every capacity and uses less energy than host-only.

H8c Dynamic vs static: $\geq$ 10% lower energy, quality loss within +2%, at default resources and the bottom-up migration cost.

H9a Reusable (dense): dynamic beats static by $\geq$ 10% energy at neutral quality.

H9b Streaming: no move can be repaid; dynamic $\approx$ static (energy difference $< 2\%$).

H9c Random access at $10\times$ cost: ungated dynamic is worse than static; the gated policy is within +1% of min(static, ungated).

H9d The best of {flat, static, dynamic, dynamic+gate} differs across regimes, and dynamic+gate is never more than 2% worse than static in energy.

H9e Break-even migration scale in the explicit model (reported; no threshold pre-set).

Outcomes are in Tab. 8: H9c and the second half of H9d were *not* met.

C Full result tables

Table 9: E8 in full: per-trace ratios to flat, averaged over 42 traces, by CIM capacity. T = compute in memory (tier), H = host only, A = adaptive spill. Last column: share of WARM packets the adaptive scheduler spilled. Flat: mean makespan 5104 μs, mean energy 3681 μJ.

	Makespan / flat			Energy / flat			Bus bytes / flat			
CIM units	T	H	A	T	H	A	T	H	A	Spill (A)
Static modality										
2	4.984	0.923	0.875	0.108	0.319	0.282	0.085	0.313	0.273	83%
4	2.504	0.923	0.762	0.108	0.319	0.255	0.085	0.313	0.244	70%
8	1.252	0.923	0.601	0.108	0.319	0.219	0.085	0.313	0.205	53%
16	0.637	0.923	0.424	0.108	0.319	0.179	0.085	0.313	0.162	34%
32	0.330	0.923	0.270	0.108	0.319	0.145	0.085	0.313	0.125	18%
64	0.176	0.923	0.154	0.108	0.319	0.121	0.085	0.313	0.098	6%
Dynamic ($8\times$ /$0.4\times$)										
2	5.165	0.926	0.880	0.079	0.296	0.260	0.055	0.290	0.250	83%
4	2.591	0.926	0.767	0.079	0.296	0.233	0.055	0.290	0.221	71%
8	1.306	0.926	0.606	0.079	0.296	0.197	0.055	0.290	0.182	54%
16	0.667	0.926	0.429	0.079	0.296	0.157	0.055	0.290	0.140	36%
32	0.338	0.926	0.275	0.079	0.296	0.123	0.055	0.290	0.103	20%
64	0.184	0.926	0.160	0.079	0.296	0.098	0.055	0.290	0.076	9%

D Reproducing the results

```
python -m venv .venv && .venv/bin/pip install -e ".[dev,vlm]"
.venv/bin/python -m pytest -q                       # 77 tests, incl. legacy-equivalence of system.py
.venv/bin/python experiments/make_dataset.py        # 42-image set (COCO photos re-downloaded)
.venv/bin/python experiments/extract_many.py        # real attention traces (MPS/GPU)
.venv/bin/python experiments/run_m7.py s256         # E5 grid, 256M      (analyze_m7.py for tables)
.venv/bin/python experiments/run_m7_estimators.py   # E6
.venv/bin/python experiments/validate_real_output.py 42 mps   # E7
.venv/bin/python experiments/run_m9.py e8 e9 e9b e9c          # E8-E9 (3,402 simulations)
.venv/bin/python experiments/analyze_m9.py          # summaries in results/m9_*_summary.csv
.venv/bin/python paper/make_figures.py && .venv/bin/python paper/make_tables.py
cd paper && pdflatex main.tex && pdflatex main.tex
```

The simulations of E8–E9 take about 25 minutes on 7 cores; the lab notebook `NOTES.md` records every deviation and every pre-specified criterion in chronological order.

Table 10: E9 in full: change against static modality (%, mean over 42 traces, paired-bootstrap 95% CI in brackets). Scale = migration cost relative to the bottom-up estimate. Moves per session; bus bytes and makespan as % change; Win = fraction of traces with lower energy and quality loss at most 2% worse. † Post-hoc variants (see text); streaming rows for gated variants were not run (the dynamic policy makes no moves there).

Policy	Scale	Energy	Quality loss	Moves	Bus bytes	Makespan	Win
Reusable (dense)							
Flat (headroom)	–	9.25× static	–	–	–	–	–
Dynamic	1×	-27.4 [-27.8, -27.0]	-2.5 [-4.0, -1.0]	1190	-35.6	+4.7	0.88
Dynamic	10×	-21.6 [-22.0, -21.1]	-2.5 [-4.0, -1.1]	1190	-35.6	+13.1	0.88
+ reuse gate	1×	-27.4 [-27.8, -27.0]	-2.5 [-4.0, -1.0]	1190	-35.6	+4.7	0.88
+ reuse gate	10×	-21.6 [-22.0, -21.1]	-2.5 [-4.0, -1.1]	1190	-35.6	+13.1	0.88
+ gate, min age 8†	1×	-23.1 [-23.6, -22.5]	-5.4 [-6.7, -4.3]	1136	-30.0	+4.2	1.00
+ gate, min age 8†	10×	-17.5 [-18.0, -17.0]	-5.4 [-6.7, -4.2]	1136	-30.0	+13.0	1.00
+ gate, min age 16†	1×	-20.0 [-20.5, -19.5]	-4.0 [-5.1, -3.0]	1115	-26.0	+3.7	1.00
+ gate, min age 16†	10×	-14.6 [-15.0, -14.1]	-4.0 [-5.1, -3.0]	1115	-26.0	+11.9	1.00
Random 30%							
Flat (headroom)	–	9.07× static	–	–	–	–	–
Dynamic	1×	-25.1 [-25.4, -24.7]	+3.9 [+3.0, +4.8]	1038	-32.2	+4.4	0.14
Dynamic	10×	-8.5 [-8.7, -8.3]	+3.9 [+2.9, +4.8]	1038	-32.2	+44.0	0.14
+ reuse gate	1×	-25.1 [-25.4, -24.7]	+3.9 [+3.0, +4.8]	1038	-32.2	+4.4	0.14
+ reuse gate	10×	-5.4 [-5.5, -5.2]	+2.0 [+1.4, +2.5]	744	-22.1	+34.1	0.45
+ gate, min age 4†	1×	-24.9 [-25.2, -24.5]	+3.7 [+2.8, +4.6]	1035	-31.9	+3.8	0.14
+ gate, min age 4†	10×	-5.2 [-5.3, -5.1]	+1.8 [+1.3, +2.4]	741	-21.8	+28.0	0.48
+ gate, min age 8†	1×	-23.6 [-23.9, -23.2]	+2.8 [+2.0, +3.7]	1021	-30.2	+2.9	0.29
+ gate, min age 8†	10×	-3.6 [-3.7, -3.5]	+0.7 [+0.2, +1.1]	687	-18.7	+23.1	0.86
+ gate, min age 16†	1×	-19.6 [-19.9, -19.2]	+1.0 [+0.4, +1.6]	983	-25.0	+1.5	0.71
+ gate, min age 16†	10×	-0.8 [-0.9, -0.7]	-0.1 [-0.5, +0.2]	586	-13.0	+22.7	1.00
Random 10%							
Flat (headroom)	–	8.58× static	–	–	–	–	–
Dynamic	1×	-15.8 [-16.0, -15.5]	+5.7 [+5.2, +6.2]	1064	-19.1	+18.6	0.00
Dynamic	10×	+32.1 [+31.7, +32.6]	+5.7 [+5.2, +6.2]	1064	-19.1	+186.2	0.00
+ reuse gate	1×	-15.8 [-16.0, -15.5]	+5.7 [+5.2, +6.2]	1059	-19.1	+18.5	0.00
+ reuse gate	10×	+2.2 [+2.1, +2.2]	+0.5 [+0.4, +0.6]	91	-2.4	+15.7	0.00
+ gate, min age 4†	1×	-15.8 [-16.0, -15.5]	+5.7 [+5.2, +6.2]	1059	-19.1	+18.5	0.00
+ gate, min age 4†	10×	+2.1 [+2.1, +2.2]	+0.5 [+0.4, +0.6]	91	-2.4	+15.5	0.00
+ gate, min age 8†	1×	-15.5 [-15.8, -15.3]	+5.5 [+5.1, +6.0]	1057	-18.8	+16.4	0.00
+ gate, min age 8†	10×	+1.1 [+1.0, +1.1]	+0.1 [+0.1, +0.2]	42	-1.0	+7.3	0.00
+ gate, min age 16†	1×	-14.3 [-14.5, -14.0]	+4.8 [+4.4, +5.2]	1048	-17.2	+11.1	0.00
+ gate, min age 16†	10×	+0.1 [+0.1, +0.1]	+0.0 [-0.0, +0.0]	4	-0.1	+0.8	0.00
Random 3%							
Flat (headroom)	–	7.25× static	–	–	–	–	–
Dynamic	1×	+2.1 [+2.0, +2.2]	+2.6 [+2.4, +2.9]	645	+4.3	+35.6	0.00
Dynamic	10×	+81.9 [+80.5, +83.3]	+2.6 [+2.4, +2.8]	645	+4.3	+353.7	0.00
+ reuse gate	1×	+1.5 [+1.4, +1.6]	+2.6 [+2.4, +2.8]	598	+3.4	+33.0	0.00
+ reuse gate	10×	+1.1 [+1.0, +1.2]	+0.1 [+0.0, +0.1]	9	-0.0	+5.2	0.00
+ gate, min age 4†	1×	+1.5 [+1.4, +1.6]	+2.6 [+2.4, +2.8]	598	+3.4	+33.0	0.00
+ gate, min age 4†	10×	+1.1 [+1.0, +1.2]	+0.1 [+0.0, +0.1]	9	-0.0	+5.2	0.00
+ gate, min age 8†	1×	+1.5 [+1.4, +1.6]	+2.6 [+2.4, +2.8]	598	+3.5	+33.0	0.00
+ gate, min age 8†	10×	+0.1 [+0.1, +0.2]	+0.0 [+0.0, +0.0]	1	-0.0	+0.7	0.00
+ gate, min age 16†	1×	+1.7 [+1.6, +1.8]	+2.4 [+2.2, +2.7]	596	+3.6	+30.7	0.00
+ gate, min age 16†	10×	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]	0	+0.0	+0.0	0.00
Streaming							
Flat (headroom)	–	7.28× static	–	–	–	–	–
Dynamic	1×	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]	0	+0.0	+0.0	0.00
Dynamic	10×	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]	0	+0.0	+0.0	0.00
+ reuse gate	1×	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]	0	+0.0	+0.0	0.00
+ reuse gate	10×	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]	0	+0.0	+0.0	0.00